

Documentación Técnica Comparativa: Web Vulnerable y Web Segura

Laboratorio de Seguridad Web

24 de julio de 2026

Índice

1. Objetivo del documento	2
2. Descripción general del proyecto	2
3. Entorno de ejecución	2
4. Comparación general de seguridad	2
5. Análisis paso a paso de vulnerabilidades y mejoras	3
5.1. Paso 1: Inyección SQL en el inicio de sesión	3
5.2. Paso 2: Autenticación débil y contraseñas en texto plano	4
5.3. Paso 3: Protección de sesión	4
5.4. Paso 4: IDOR y exposición de perfiles ajenos	4
5.5. Paso 5: Control de acceso roto en el panel administrativo	5
5.6. Paso 6: Cross-Site Scripting en búsquedas y productos	5
5.7. Paso 7: Falta de protección CSRF	6
5.8. Paso 8: XXE en importación XML	6
5.9. Paso 9: Deserialización insegura	7
5.10. Paso 10: Configuración insegura y exposición de información	7
5.11. Paso 11: Componentes vulnerables y cabeceras de seguridad	8
5.12. Paso 12: Falta de registros y monitoreo	8
6. Resumen de mejoras aplicadas	8
7. Conclusiones	9
8. Recomendaciones finales	9

1. Objetivo del documento

El objetivo de esta documentación es explicar, paso a paso, las diferencias técnicas entre dos aplicaciones PHP del mismo laboratorio: **web vulnerable** y **web seguro**. La primera aplicación fue construida intencionalmente con fallas de seguridad basadas en OWASP Top 10 2017. La segunda conserva la misma idea funcional de tienda de productos, pero corrige esas debilidades mediante controles defensivos.

Este documento no se limita a decir que una aplicación es vulnerable y la otra segura. Explica qué problema existía, por qué era peligroso y qué mejora concreta se aplicó en la versión segura.

2. Descripción general del proyecto

El proyecto contiene dos implementaciones separadas:

- **web vulnerable**: aplicación llamada *VulnShop*, diseñada como laboratorio de práctica ofensiva. Incluye fallas como inyección SQL, autenticación débil, exposición de datos sensibles, XXE, control de acceso roto, XSS, deserialización insegura y falta de monitoreo.
- **web seguro**: aplicación llamada *SecureShop*, diseñada como contraparte segura. Implementa consultas preparadas, validación de entradas, hash de contraseñas, protección CSRF, control de roles, codificación de salida, cabeceras HTTP de seguridad y registro de eventos.

3. Entorno de ejecución

Ambas aplicaciones usan PHP 8 y MySQL o MariaDB. Sin embargo, cada una usa una base de datos distinta para mantener separados los escenarios de prueba.

Elemento	Web vulnerable	Web segura
Nombre de aplicación	VulnShop	SecureShop
Base de datos	vulnshop_db	seureshop_db
Usuario de base de datos	vulnshop_user	seureshop_user
Puerto sugerido	127.0.0.1:8000	127.0.0.1:8080
Propósito	Laboratorio vulnerable	Versión defensiva

4. Comparación general de seguridad

Área	Problema en web vulnerable	Mejora en web segura
Consultas SQL	Se concatena entrada del usuario directamente en las consultas.	Se usan consultas preparadas con parámetros.

Autenticación	Las contraseñas se guardan y comparan en texto plano.	Se usa <code>password_hash()</code> y <code>password_verify()</code> .
Sesiones	No se regenera el identificador de sesión al iniciar sesión.	Se ejecuta <code>session_regenerate_id(true)</code> después del login exitoso.
Control de acceso	El rol puede manipularse desde la URL.	El rol se valida desde la sesión del servidor.
Datos sensibles	Se muestran contraseñas y tarjetas completas.	Se ocultan contraseñas y se enmascaran tarjetas.
XSS	Se imprime contenido sin escapar.	Se centraliza la codificación con la función <code>e()</code> .
CSRF	Los formularios críticos no tienen token CSRF.	Los formularios usan <code>csrf_field()</code> y <code>verify_csrf()</code> .
XXE	El parser XML permite entidades externas.	Se bloquea expansión de entidades y acceso de red con <code>LIBXML_NONET</code> .
Deserialización	Se ejecuta <code>unserialize()</code> sobre entrada del usuario.	Se reemplaza por JSON con campos permitidos.
Monitoreo	No se registran eventos de seguridad.	Se registran eventos en <code>security_logs</code> .

5. Análisis paso a paso de vulnerabilidades y mejoras

5.1. Paso 1: Inyección SQL en el inicio de sesión

Problema en la web vulnerable

En `web_vulnerable/login.php`, el correo y la contraseña ingresados por el usuario se insertan directamente dentro de la consulta SQL:

```
$sql = "SELECT_*_FROM_users_WHERE_email=_'$email' _AND_password=_'$password'";
$user = db()->query($sql)->fetch(PDO::FETCH_ASSOC);
```

Esto permite que un atacante modifique la lógica de la consulta. Por ejemplo, si el campo de correo recibe una condición SQL maliciosa, la base de datos puede interpretar esa entrada como parte del comando y no como un simple dato.

Mejora en la web segura

En `web_seguro/login.php`, la consulta se realiza con `prepare()` y parámetros:

```
$stmt = db()->prepare('SELECT_*_FROM_users_WHERE_email=_?_LIMIT_1');
$stmt->execute([$email]);
```

Con este cambio, la entrada del usuario se trata como dato y no como código SQL. Esta es una mejora fundamental porque separa la estructura de la consulta de los valores ingresados.

5.2. Paso 2: Autenticación débil y contraseñas en texto plano

Problema en la web vulnerable

En la versión vulnerable, la tabla `users` guarda la contraseña directamente en una columna llamada `password`. Además, el login compara el valor ingresado con el texto almacenado.

```
('Admin_User', 'admin@example.com', 'admin123', 'admin', ...)
```

Esto es peligroso porque si la base de datos se filtra, las contraseñas quedan expuestas inmediatamente.

Mejora en la web segura

La versión segura usa una columna `password_hash` y genera hashes con `password_hash()`:

```
password_hash($password, PASSWORD_DEFAULT)
```

Luego valida el login con:

```
password_verify($password, $user['password_hash'])
```

Esta mejora impide que una filtración de la base de datos revele directamente las contraseñas reales.

5.3. Paso 3: Protección de sesión

Problema en la web vulnerable

Después del login, la versión vulnerable asigna datos a la sesión, pero no regenera el identificador de sesión:

```
$_SESSION['user_id'] = $user['id'];  
$_SESSION['role'] = $user['role'];
```

Esto deja abierta la posibilidad de ataques de fijación de sesión si un atacante logra imponer o conocer un identificador anterior.

Mejora en la web segura

La versión segura regenera la sesión después del login exitoso:

```
session_regenerate_id(true);
```

También configura la cookie de sesión con `httponly` y `samesite=Lax` en `web_seguro/config.php`. Esto reduce el riesgo de robo o abuso de la cookie desde JavaScript o desde solicitudes externas.

5.4. Paso 4: IDOR y exposición de perfiles ajenos

Problema en la web vulnerable

En `web_vulnerable/profile.php`, el identificador del usuario puede venir desde la URL:

```
$id = $_GET['id'] ?? current_user_id();  
$user = db()->query("SELECT * FROM users WHERE id = $id")->fetch(PDO::  
    FETCH_ASSOC);
```

Esto produce una vulnerabilidad IDOR, porque un usuario autenticado puede cambiar el parámetro `id` y consultar el perfil de otra persona. Además, la página muestra contraseña, rol, tarjeta completa y dirección.

Mejora en la web segura

En `web_seguro/profile.php`, el perfil siempre se obtiene usando el usuario autenticado en sesión:

```
$stmt = db()->prepare('SELECT id, name, email, role, credit_card_last4, address  
    FROM users WHERE id = ?');  
$stmt->execute([current_user_id()]);
```

La mejora elimina la dependencia del parámetro `id`. Además, no se muestra la contraseña y la tarjeta se presenta enmascarada.

5.5. Paso 5: Control de acceso roto en el panel administrativo

Problema en la web vulnerable

En `web_vulnerable/admin.php`, el rol puede venir desde el parámetro `role` de la URL:

```
$role = $_GET['role'] ?? current_user_role();
```

Esto significa que un usuario puede intentar acceder como administrador agregando `?role=admin`. El problema de fondo es que la aplicación confía en un dato controlado por el cliente.

Mejora en la web segura

En `web_seguro/admin.php`, el acceso se protege con:

```
require_admin();
```

La función `require_admin()` valida el rol guardado en la sesión del servidor y devuelve código 403 Forbidden si el usuario no es administrador. También registra el intento con `log_security_event()`.

5.6. Paso 6: Cross-Site Scripting en búsquedas y productos

Problema en la web vulnerable

En `web_vulnerable/search.php`, la búsqueda se imprime directamente en el HTML:

```
<p>Results for: <?= $q ?></p>
```

También se imprimen nombres y descripciones de productos sin codificación. Si un atacante logra insertar JavaScript, el navegador lo puede ejecutar.

Mejora en la web segura

La versión segura define una función central para escapar salidas HTML:

```
function e($value)
{
    return htmlspecialchars((string) $value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

Luego imprime datos con `e()`:

```
<p>Results for: <?= e($q) ?></p>
```

Esta mejora evita que el navegador interprete contenido del usuario como código HTML o JavaScript.

5.7. Paso 7: Falta de protección CSRF

Problema en la web vulnerable

Los formularios sensibles de la versión vulnerable, como login, creación de productos o importación XML, no validan un token CSRF. Esto permite que otro sitio intente enviar solicitudes en nombre del usuario si este tiene una sesión activa.

Mejora en la web segura

La versión segura genera y valida tokens CSRF mediante `csrf_field()` y `verify_csrf()`:

```
<?= csrf_field() ?>
verify_csrf();
```

El token se guarda en sesión y se compara con `hash_equals()`, evitando comparaciones inseguras.

5.8. Paso 8: XXE en importación XML

Problema en la web vulnerable

En `web_vulnerable/import_xml.php`, el parser XML permite cargar DTD y expandir entidades externas:

```
$dom->loadXML($xmlText, LIBXML_NOENT | LIBXML_DTDLOAD);
```

Esto permite ataques XXE. Un atacante podría intentar leer archivos locales del servidor usando entidades externas.

Mejora en la web segura

En `web_seguro/import_xml.php`, la carga XML usa banderas más restrictivas:

```
$loaded = $dom->loadXML($xmlText, LIBXML_NONET | LIBXML_NOERROR |
    LIBXML_NOWARNING);
```

Además, el importador queda restringido a usuarios administradores y protegido con CSRF. La mejora reduce el riesgo de lectura de archivos locales o solicitudes externas desde el parser XML.

5.9. Paso 9: Deserialización insegura

Problema en la web vulnerable

En `web_vulnerable/deserialize.php`, la aplicación ejecuta `unserialize()` sobre datos enviados por el usuario:

```
$result = print_r(unserialize($payload), true);
```

El archivo también define una clase `FileWriter` con un método `__destruct()` que escribe archivos. Esto es riesgoso porque un atacante puede construir objetos serializados para provocar efectos secundarios.

Mejora en la web segura

La versión segura elimina el uso de `unserialize()` para entrada del usuario y usa JSON con una lista de campos permitidos en `web_seguro/import_json.php`:

```
$data = json_decode($payload, true, 512, JSON_THROW_ON_ERROR);
$allowed = [
    'name' => isset($data['name']) ? (string) $data['name'] : '',
    'price' => isset($data['price']) ? (float) $data['price'] : 0,
];
```

La mejora consiste en aceptar datos simples y conocidos, no objetos PHP con comportamiento ejecutable.

5.10. Paso 10: Configuración insegura y exposición de información

Problema en la web vulnerable

En `web_vulnerable/config.php`, la aplicación muestra errores y mantiene el modo debug activo:

```
ini_set('display_errors', '1');
define('DEBUG_MODE', true);
define('APP_SECRET', 'hardcoded-secret-key-123');
```

Además, existe una ruta `debug.php`, un archivo `robots.txt` y una copia `backup/users.sql.bak`, lo cual facilita la enumeración y exposición de datos durante el laboratorio.

Mejora en la web segura

En `web_seguro/config.php`, los errores no se muestran directamente al usuario:

```
ini_set('display_errors', '0');
```

La versión segura también evita exponer una ruta de debug pública y agrega cabeceras HTTP defensivas desde `web_seguro/includes/header.php`.

5.11. Paso 11: Componentes vulnerables y cabeceras de seguridad

Problema en la web vulnerable

La web vulnerable carga una versión antigua de jQuery desde CDN:

```
<script src="https://code.jquery.com/jquery-1.12.4.min.js"></script>
```

Usar dependencias antiguas aumenta el riesgo de vulnerabilidades conocidas. Además, la aplicación no define cabeceras de seguridad importantes.

Mejora en la web segura

La web segura no carga esa dependencia antigua y define cabeceras como:

```
header('X-Frame-Options:␣DENY');  
header('X-Content-Type-Options:␣nosniff');  
header('Referrer-Policy:␣strict-origin-when-cross-origin');  
header("Content-Security-Policy:␣default-src␣'self';␣...");
```

Estas cabeceras ayudan a reducir riesgos como clickjacking, interpretación incorrecta de tipos MIME y ejecución de recursos no autorizados.

5.12. Paso 12: Falta de registros y monitoreo

Problema en la web vulnerable

La web vulnerable no registra eventos importantes como intentos fallidos de login, acceso denegado o acciones administrativas. Esto dificulta detectar ataques o investigar incidentes.

Mejora en la web segura

La web segura incorpora la función `log_security_event()`:

```
$stmt = db()->prepare('INSERT␣INTO␣security_logs␣(user_id,␣event_type,␣message,␣  
ip_address,␣created_at)␣VALUES␣(?,␣?,␣?,␣?,␣NOW())');
```

También incluye `security_logs.php`, una vista administrativa para revisar los últimos eventos. Esta mejora no evita todos los ataques, pero permite detectarlos y responder mejor.

6. Resumen de mejoras aplicadas

Mejora	Impacto técnico
Consultas preparadas	Evitan que la entrada del usuario modifique la consulta SQL.
Hash de contraseñas	Reduce el impacto de una filtración de base de datos.
Regeneración de sesión	Disminuye el riesgo de fijación de sesión.

Validación de roles en servidor	Impide que el usuario escale privilegios modificando la URL.
Codificación de salida	Mitiga XSS reflejado y almacenado.
Tokens CSRF	Evitan acciones no autorizadas enviadas desde otros sitios.
XML seguro	Reduce el riesgo de XXE y lectura de archivos locales.
JSON con allow-list	Reemplaza la deserialización insegura por datos simples y controlados.
Cabeceras HTTP de seguridad	Agregan defensa en profundidad a nivel de navegador.
Logs de seguridad	Permiten detectar intentos sospechosos y auditar acciones críticas.

7. Conclusiones

La diferencia principal entre ambas aplicaciones no está en la funcionalidad visible, sino en la forma en que procesan datos, autentican usuarios y toman decisiones de autorización. La web vulnerable confía demasiado en entradas controladas por el usuario, muestra información sensible y ejecuta operaciones peligrosas sin validación suficiente. La web segura corrige esos errores aplicando controles básicos pero esenciales.

Las mejoras más importantes son el uso de consultas preparadas, hash de contraseñas, validación de sesión y rol en el servidor, codificación de salida, protección CSRF, configuración segura del parser XML, eliminación de deserialización insegura y registro de eventos de seguridad.

Como aprendizaje central, una aplicación segura no se construye agregando una sola defensa. Se construye aplicando varias capas: validación de entrada, codificación de salida, autenticación sólida, autorización estricta, configuración segura y monitoreo. Cada capa reduce una parte del riesgo.

8. Recomendaciones finales

- Mantener la web vulnerable solo en entornos locales de práctica y nunca publicarla en internet.
- Usar la web segura como referencia para implementar defensas mínimas en aplicaciones PHP.
- Revisar periódicamente dependencias, configuración del servidor y permisos de archivos.
- Complementar estas defensas con pruebas automatizadas de seguridad y revisión de código.
- No guardar secretos reales directamente en el código fuente; usar variables de entorno o gestores de secretos en entornos reales.